

Bakir, Abdil Karim

Software Engineering · Backend, Web, Cloud, Hardware



PHOTOGRAPH

AVAILABILITY	Immediately · working student, part-time
LOCATION	Berlin, Germany
EMAIL	abdilkarimb@gmail.com
GITHUB	github.com/KarimBk7
ONLINE VERSION	my-website.abdilkarimb.workers.dev/en
AS OF	28 September 2026

368

commits of my own in the client project
of 703 in the repository — backend architect in
a team of six

258

tests, green on every push
ReflowTask: 225 in the backend, 33 in the
frontend

5

programs shipped
from a Windows installer to a Docker stack on
my own hardware

CONTENTS

M-1	Expense configurator for Projektron BCS · Delivered 07/2026
M-2	ReflowTask · In service v1.0
M-3	This page · Published 09/2026
M-4	Kaiju Adventure · Submitted 12/2022
M-5	Technical learning repository · Accepted 67/67
M-6	Wordle Solver · Released v1.0

This document is generated from the application site and carries the
same revision. Videos and the interactive demo exist online only;
the document shows a still frame instead. Every linked address is
written out in full.

Expense configurator for Projektron BCS

A Windows desktop application that lets HR and accounting teams maintain the CSV-based travel expense configuration of Projektron BCS without knowing the underlying file format.

CLIENT	Projektron GmbH
CONTEXT	Software project, FU Berlin
PERIOD	13 Apr – 1 Aug 2026
TEAM	6 people
MY ROLE	Backend Architect

TASK

The configuration defines, for every combination of work and travel country, which allowances, thresholds and reduction rules apply in which period. Maintained by hand it was opaque and error-prone — Excel turns monetary values into dates on open and offers no domain validation.

The core technical problem

JavaFX cannot pin individual columns. The frozen left column is therefore a second table, kept in sync through the pixel offset of the virtualised view. The stress test with 10,000 rows then showed that the validation logic was not the slow part — its feedback to the table was: one list observer fired around 9,600 individual UI events. Replacing it with a single signal per validation pass produced the factor of 34.

MY CONTRIBUTION

Table core built from two pixel-synchronised tables with a frozen column

Multi-tab handling with fully encapsulated state per file

Undo/redo on the memento pattern with a file-size dependent memory budget

Runtime language switching

Crash protection with automatic recovery of the working state

Performance work following the 10,000-row stress test

368

own commits
of 703 in the repository

3

memory leaks fixed
found in code review

19

test classes
plus drivers for load and UI

90+

classes
in the delivered source

PERFORMANCE WORK

A large part of my work went into performance. After the 10,000-row stress test I measured every optimisation individually, before and after; the four most important values are here. They come from the project documentation, not from memory.

TEST POINT	BEFORE	AFTER
Validation after every edit 10,000 rows	5,051 ms	148 ms Factor 34×
Load and render the file 10,000 rows	5,000 ms	600 ms Factor ~8×
Validation report with 10,000 errors 10,000 rows	SEVERAL MINUTES	283 ms
UI events per validation pass 10,000 rows	9,600	1 Factor 9600 : 1

Measured with a TestFX driver that boots the real UI and injects 10,000 rows into the live table – not a synthetic setup. Reproducible via `mvn test -Dtest=PerfVerifyDriver`.

THE CHANGE

Four lines fewer, one signal more. Before that was clear I measured layer by layer where the time went — it was not the validation logic.

plain list	142 ms
+ extractor	250 ms
+ filtered/sorted	260 ms
+ TableView	5,051 ms

95 % of the cost was the table reacting to some 9,600 individual events – not the validation itself.

BEFORE

```
// Extractor tells JavaFX to fire list-change
// events when hasRowError/hasRowWarning change
private final ObservableList<ExpenseEntry> data =
    FXCollections.observableArrayList(
        entry -> new Observable[] {
            entry.hasRowError,
            entry.hasRowWarning});
```

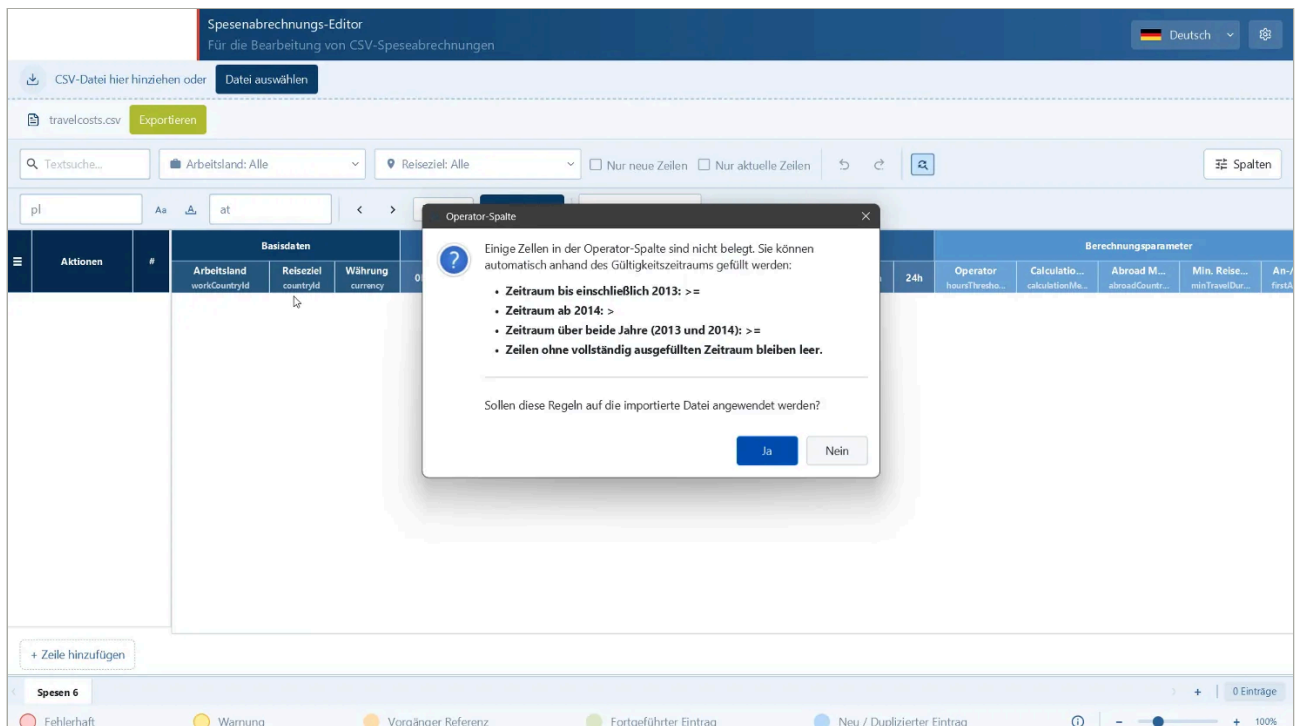
AFTER

```
private final ObservableList<ExpenseEntry> data =
    FXCollections.observableArrayList();

// Bumped once after each full validation pass
private final IntegerProperty validationEpoch =
    new SimpleIntegerProperty(0);

public void markValidated() {
    validationEpoch.set(validationEpoch.get() + 1);
}
```

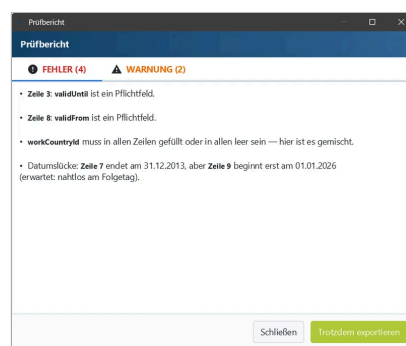
ExpenseManager.java · Commit f4e8223



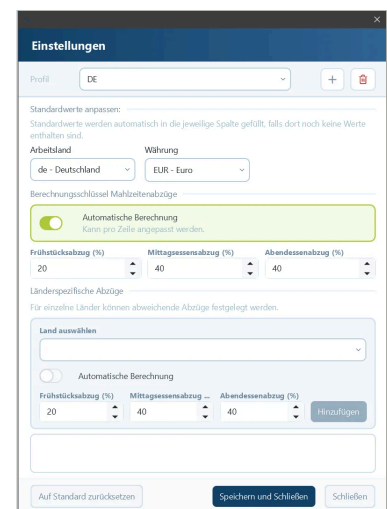
Live use, 24 seconds: after the import, a dialog offers to fill empty operator cells from the validity period. Then filtering by destination, find and replace, editing a cell, a row flagged yellow as a warning, scrolling through the column groups, the settings and saving.



Row wizard: guided entry field by field, with country-dependent steps.



Validation report: collects every finding and jumps to the cell on click.



Settings: default values per country, storable as named profiles.

What I would build differently today

Three entries from my own decision log. They were recorded during the project as deliberate trade-offs — not arranged after the fact.

The frozen column depends on `fixedCellSize`. That is fast but brittle: after structural and column changes cells stay unpositioned until a one-pixel scroll nudges the view. A deliberate workaround, not a design. (D3)

Undo deep-copies the entire row list on every action, about 2.7 KB per row. I assessed a delta-based undo and rejected it as too costly — then had to retrofit the memory bound after all. (D5, corrected in D22)

The dialog helpers in `util` import their controllers directly. An interface each would be cleaner; I judged that over-engineering for FXML dialogs. (D10)

STACK

Java 21	JavaFX 21	FXML	Maven	JUnit 5
TestFX	CSS	jpackage	GitHub Actions	Spotless

DELIVERY

Delivered as a native Windows installer with a bundled Java runtime, installable without a separate Java installation and without administrator rights. Shipped alongside: a decision log, a validation rule catalogue and eight how-to guides.

CONTEXT

The functions listed come from my area of responsibility within the six-person team. UI, prototyping and requirements were handled by the other members; the domain validation logic was developed jointly. The source code belongs to the client and is therefore the only project shown here that is not publicly viewable.

ReflowTask

A task planner that repairs its own schedule: every task gets real time blocks, and a missed block is replanned automatically. Self-hosted for a household, bilingual, on the desktop and on the phone, running on a Raspberry Pi 5.

CONTEXT	Own work
PERIOD	14–28 Sep 2026
MY ROLE	Backend, interface, operations
SCOPE	5,120 lines of Java, 6,166 of TypeScript
STATUS	Version 1.0, Raspberry Pi 5, Docker Compose

TASK Task lists tell you what to do, not when. Miss an appointment and the task turns red, leaving you to rearrange the day by hand. Hosted services that replan for you cost a subscription and keep your calendar on someone else’s servers. ReflowTask does the same thing on your own hardware: every task carries an estimated duration and an optional deadline, and from those it places blocks inside your own working hours. Planning as a pure function The core is deliberately a pure function: tasks, committed time, settings and the current moment go in, finished blocks come out — no database, no injected clock. That makes every rule provable by a single test. The order is deadline first, then priority, then the age of the task. That last tie-break is not cosmetic: without it, equivalent tasks would swap places on every run and each replan would look like a change. Free capacity also always starts on a clean quarter hour; otherwise a replan at 10:37 produces a calendar full of 10:37 starts.	MY CONTRIBUTION Planning core as a pure function, testable directly without a database An hourly job spots missed blocks and replans only what is left Per-block states (planned, done, missed): finished parts count against the estimate Repeating tasks, an earliest start, and dependencies: a task can wait for another Calendar sync both ways over ICS: outside appointments block time, and the plan itself can be subscribed to Working and personal time kept apart, plus a freeze window for whatever is about to start Household accounts with their own tasks, working hours and history per person Data isolation tested adversarially: another user’s ID answers 404, not 403 Login with session cookies, bcrypt and a lockout after five failed attempts
--	---

A real phone layout rather than a squeezed desktop,
installable as a web app

Bilingual interface, data export as JSON or CSV,
verified backups via script

Shipped as Docker Compose, running on a
Raspberry Pi 5 behind Caddy and Tailscale

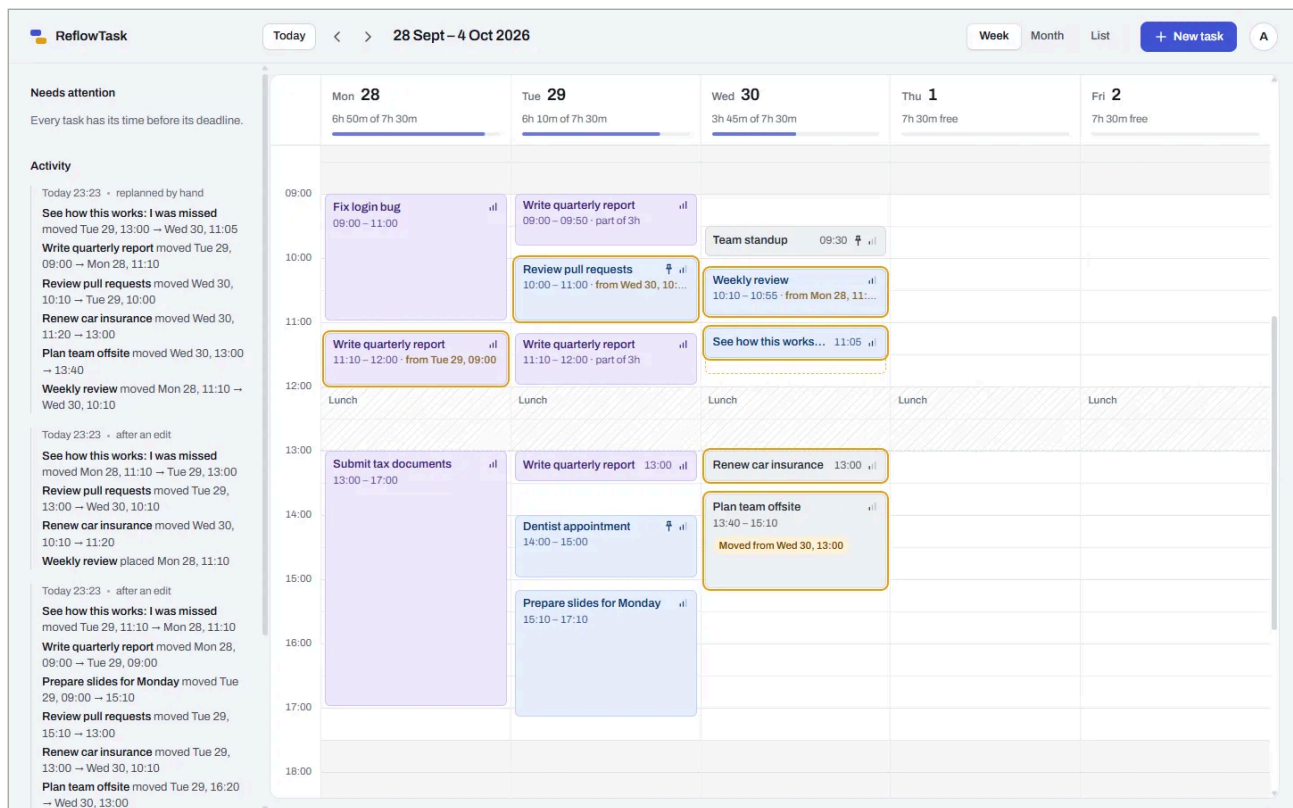
KEY FIGURES

225 backend tests, all green
run on 28 Sep 2026, plus 33 in the
frontend

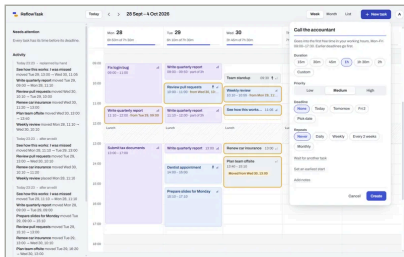
12 database migrations
run identically on H2 and
PostgreSQL

55 commits up to version 1.0
in two weeks, 14–28 Sep 2026

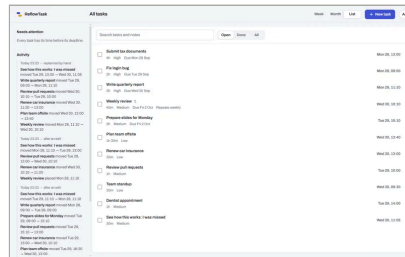
76 Java classes
plus 34 files in the frontend



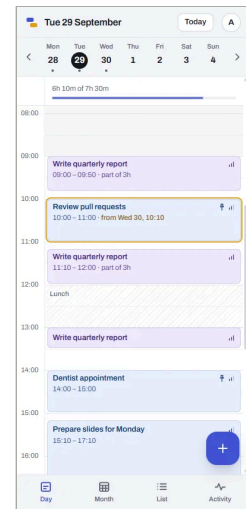
Week view: planned blocks inside the working hours, an amber outline on the ones that moved with where they came from, a dashed marker on their old slot, hatching for the lunch break. On the left, what needs attention; below it, the log of every replan and its reason. Above each day, how much of the working time is taken.



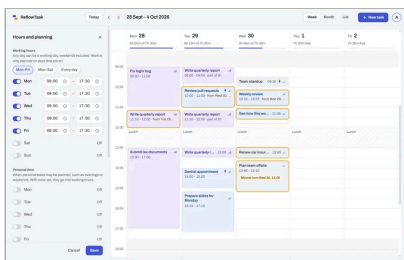
New task: duration, priority and deadline as chips instead of a form. The sentence above says in advance where the task will land.



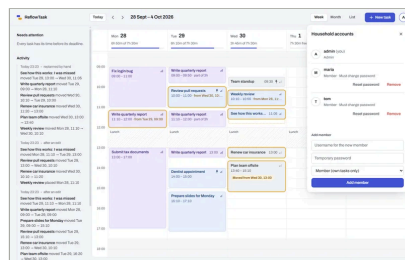
List view: search across tasks and notes, filters for open, done and all. Each row carries duration, priority, deadline and repetition — on the right, where the planner has put it.



On the phone a layout of its own rather than a squeezed desktop: one day, the week as a strip above it, the views as tabs at the bottom. Added to the home screen it starts as its own app.



Working hours per person: presets for Mon–Fri, Mon–Sat or every day, each weekday switchable on its own — shift patterns rather than a hardcoded office day. Below it, personal time: anything marked personal is planned only there.



Household accounts: add members, reset passwords, remove accounts. Each person sees only their own tasks; role and forced password change are shown on the row.

What is still missing

Version 1.0 is out and in daily use — that does not mean finished. These points are known, and they are listed here so nobody has to go looking for them.

Calendar sync is by ICS subscription, not CalDAV: other apps can see the plan but not change it, and changes only arrive on their next refresh

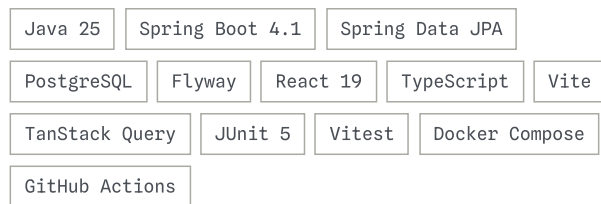
Server messages and the example task for new accounts stay English even in the German interface

Date and time fields use the browser's own inputs and therefore follow its language, not the one chosen in the app

The outline marking a moved block's old slot is a flat 30 minutes — the replan record keeps only start times

The login lockout lives in memory and is gone after a restart

STACK



DELIVERY

Version 1.0, released 28 Sep 2026. One Docker Compose command brings up PostgreSQL, the Spring Boot backend and the Nginx-served interface. It runs in production on a Raspberry Pi 5 behind Caddy, reachable over Tailscale rather than the open internet; a script takes verified, rotating database backups. A from-scratch reinstall was rehearsed to verify the setup guide.

CONTEXT

Private work; source code and interface are entirely mine. Deliberately without the full Spring Security stack: the threat model is a household on a private network, and the reasoning is written down in the repository. Passwords are reset by whoever has access to the machine — there is no mail server, so there is no reset link either.

github.com/KarimBk7/ReflowTask

(<https://github.com/KarimBk7/ReflowTask>)

This page

The application site you are reading: statically generated, bilingual, no trackers and no cookies, served as a Cloudflare Worker. One 5.2 KB script per page, no framework in the browser; sheet M-6 adds the solver as a second script.

CONTEXT	Own work
PERIOD	September 2026
MY ROLE	Design, build, operations
OPERATIONS	Cloudflare Worker
SCOPE	Two languages, one source

TASK

An application site that demonstrates what it claims: fast, no third-party services, no data collection, with content from a single verified source file instead of hand-maintained HTML. Images are generated once at build time in several sizes rather than converted at runtime.

Green locally is not green in production

Three faults reached the server despite an all-green local check: the security policy forbade inline scripts, so the page behaviour silently died; the images were requested at runtime and went nowhere; and an over-long cache lifetime served new HTML with old JavaScript. None of the three failed loudly — they simply did nothing. The answer was a second check that does not test the built site but the published one: whether the headers actually arrive, whether every image loads, and whether the script really ran.

MY CONTRIBUTION

Content in a single typed source file, both languages side by side rather than in separate files

Security policy without inline scripts, plus nosniff, referrer rule and blocked embedding

Own image pipeline: WebP at two widths at build time, never upscaled

A check against the published address, not only against the local build

Horizontal-overflow checks at 1440 and 390 pixels, including during the reveal animation

Motion fully disabled via prefers-reduced-motion

KEY FIGURES

145 ms response time
median of five requests

5,2 KB of JavaScript
one file per page, gzipped, no framework

0 trackers and cookies
no third-party servers

2 languages
complete, from one source

STACK

Astro	TypeScript	CSS	Cloudflare Workers
sharp	Playwright	GitHub Actions	

CONTEXT

Design, content and sign-off are mine; the implementation was produced in dialogue with an AI agent. The repository is open — source, check scripts and the history including the faults named above are all visible.

github.com/KarimBk7/My-Website

(<https://github.com/KarimBk7/My-Website>)

Kaiju Adventure

A complete 2D action adventure in Java, deliberately built without a game engine: own game loop, own collision detection, tile-based world and save games in a file, or optionally in a MySQL database.

CONTEXT Abitur project

PERIOD Nov – Dec 2022

MY ROLE Entire source code

TEAM Graphics and sound

SCOPE 29 classes

TASK

The goal was a finished, playable adventure without relying on a game engine — with a hand-written game loop, own rendering and own collision checks.

The game loop

A dedicated thread holds the game at 60 frames per second. Instead of fixed sleeps, a delta-time accumulator collects elapsed time and only draws once a full frame interval has accumulated. Update and draw are separated, so the game does not run faster on faster machines.

MY CONTRIBUTION

Entire source code, from rendering to database access

Game loop with delta time for a constant frame rate

Collision detection against tiles, objects and units

State machine across eleven game states

Save games via JDBC in MySQL, later switched to local files as the default

Shipped as a Windows program with its own Java runtime

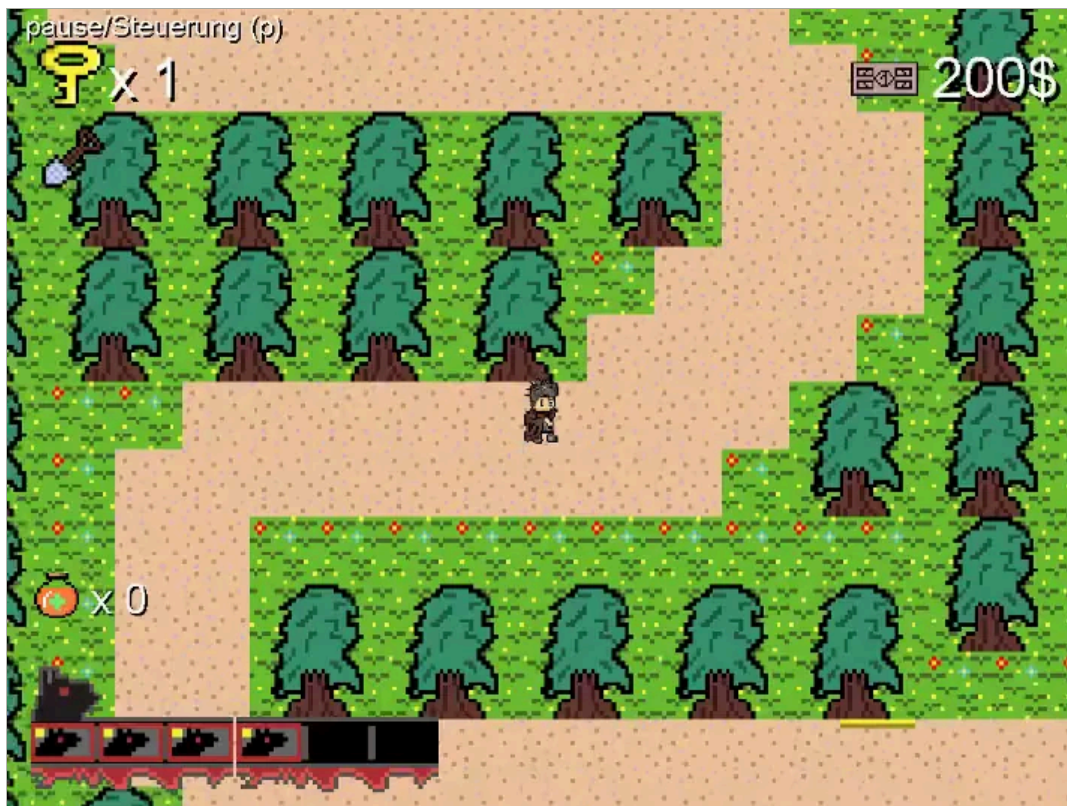
KEY FIGURES

29 Java classes
about 4,960 lines

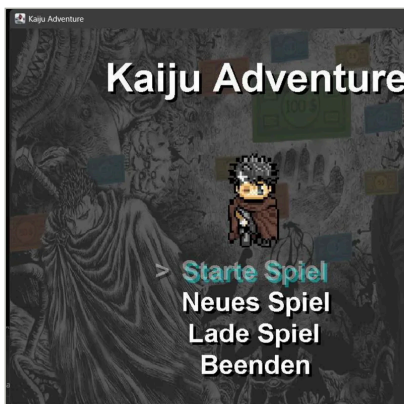
60 frames per second
own game loop

6,400 world map tiles
80 × 80 at 48 px each

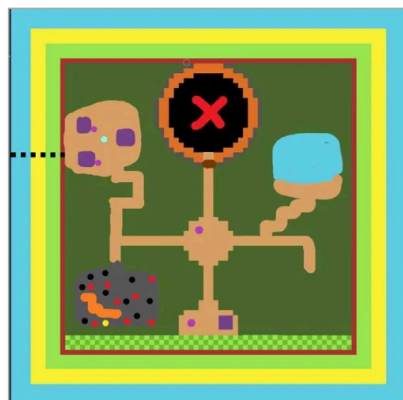
11 game states
from title to epilogue



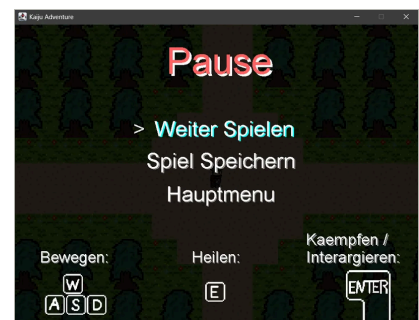
Live gameplay, 20 seconds at 60 frames per second: the camera follows the player through the forest, through the gate into the walled village and up to a villager's house. Along the way a sword strike, enemy worms, and the money counter jumping from 200 to 300 mid-run.



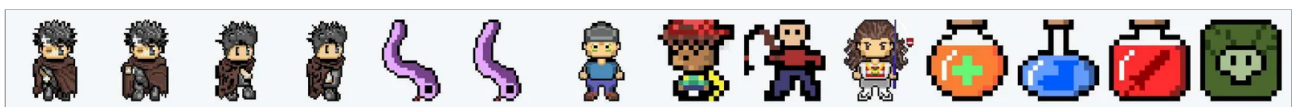
Title screen with menu navigation across the start, new game, load and quit states. Selection runs entirely through the hand-written state machine.



World design with village, lake, cave and boss arena. The grid lives as a text file in the project and is translated into tiles at startup.



Pause menu over the dimmed game world — one of the eleven game states. Resume, save and main menu, with the key bindings for moving, healing and attacking below.



Player with walk and attack frames, enemies, four non-player characters and potions. The artwork is by the team; the animation, collision and game logic behind it are mine.

Defect list, kept by myself

The project is from 2022 and was my first larger piece of software. This list is here because it shows how I read my own code today.

Database credentials are hard-coded in the source instead of held in a configuration

SQL is assembled by string concatenation instead of via parameters

The UI class grew far too large at 1,106 lines

Automated tests are missing entirely

STACK



CONTEXT

School project from 2022. The source code is entirely mine; graphics and sound came from the rest of the team.

github.com/KarimBk7/Kaiju-Adventure

(<https://github.com/KarimBk7/Kaiju-Adventure>)

Technical learning repository

A repository grown over 17 months with my own notes, terminal logs and runnable example code across 30 topic areas — from Python and Go through SQL and Linux administration to web development and testing.

CONTEXT	Programming practicum, FU Berlin
STARTED	April 2025
MY ROLE	Sole author
STATUS	67 of 67 accepted
SCOPE	30 topic areas

TASK	MY CONTRIBUTION
The programming practicum at Freie Universität Berlin replaces lectures with independent topic work. For every area you answer comprehension questions in writing, log your work at the command line, and have each assignment individually accepted by a tutor. Failed assignments are returned for revision.	My own answers in Markdown for every topic, in my own words rather than adopted solutions Terminal logs written along the way, including failed attempts and error messages Runnable example code per language, not just text excerpts Time spent per assignment recorded in the commit message

KEY FIGURES				
67 accepted assignments all submitted	163 documented hours from 140 commit messages	356 commits April 2025 to September 2026	156 own files notes, logs, code	

TOPIC MAP					
Languages4 PythonGoSQL RegExp	Tools6 Unix-Basis Unix-Diverses PaketmanagerGit Netzwerk Benutzerverwaltung	Libraries5 Standardbibliothekpip FrameworksPandas Dokumentation	Web4 HTMLCSSJavaScript HTTP		
Debugging3 DefektartenDenkweise Werkzeuge	Basics4 Unix-UmgebungRepo IDESchluss	Testing2 Testgrundlagen Unittests	Practice2 Python-mLhLinkchecker		

STACK

Python	Go	SQL	JavaScript	HTML / CSS	Linux
Bash	Git	pytest	Pandas	FastAPI	venv / pip

CONTEXT

All content is mine; third-party contributions in the repository are limited to the tutor ticking off the acceptance status. I still use it as a personal reference.

github.com/KarimBk7/Learning-repository

(<https://github.com/KarimBk7/Learning-repository>)

Wordle Solver

A small Windows program that filters every word still possible from the colour hints of a Wordle round — using the game’s own word lists, written in Python without a single dependency.

CONTEXT	Own work
PERIOD	Dec 2025 – Sep 2026
MY ROLE	Entire source code
SCOPE	367 lines of Python, three files
STATUS	Release v1.0 on GitHub

TASK After each guess Wordle gives three kinds of hint: right letter in the right place, right letter in the wrong place, and letter not in the word at all. Combining those in your head is tedious and error-prone. The program takes them as input and shows exactly the words that remain. From generating to filtering The first version built every conceivable string from the allowed letters and then checked which of them were in the word list. That grows exponentially with the number of open positions. The current version turns it around: it walks the word list once and tests each word against the constraints — fixed letters, excluded positions, allowed letters, and the required count per yellow letter. The work now depends only on the length of the list, not on the number of possibilities.	MY CONTRIBUTION Filter logic across both word lists, including a per-letter count check Tkinter interface: five columns for the five positions, a keyboard for excluding grey letters Results sorted by number of distinct letters — the next guess then rules out the most Possible solutions kept separate from words that are only accepted as guesses Shipped as a Windows folder built with PyInstaller, runnable without Python
---	---

KEY FIGURES			
14,855 words checked 2,315 solutions, 12,540 guess words	2 ms per search median of 15 runs, Python 3.14	367 lines of Python interface, solver, entry point	0 dependencies standard library only

	1st letter	2nd letter	3rd letter	4th letter	5th letter
Green (correct position)					
Can it appear again at another position?	☐ Yes ☐ No	☐ Yes ☐ No	☐ Yes ☐ No	☐ Yes ☐ No	☐ Yes ☐ No
Yellow (wrong position)					
How often in the word? 1 = exactly 1x 1+ = at least 1x					
Grey (not in word) click to exclude	Q W E R T Y U I O P A S D F G H J K L Z X C V B N M				
Find words					

Input window: one column per position in the word. The green letter at the top, below it whether it may occur again, then the yellow letters with their count, and at the bottom the keyboard for the grey ones.

Possible solutions (56)				Other allowed guesses (176)				
black	blade	bland	blank	beals	bialy	blads	blady	blaes
blast	blaze	chalk	clang	blags	blahs	blain	blanc	blase
clank	clash	clasp	dealt	blash	blate	blats	blaud	blawn
exalt	flack	flake	flaky	blaws	blays	chals	chawl	clade
flank	flash	flask	glade	clads	claes	clags	clans	claps
gland	glaze	place	plaid	clapt	clast	clats	claut	clave
plain	plait	plane	plank	clavi	claws	clays	coals	coaly
plant	plate	quail	scald	deals	dhals	dials	duals	dwale
scale	scalp	scaly	shale	elain	eland	elans	feals	flabs
shalt	shawl	slack	slain	flags	flaks	flane	flans	flaps
slang	slant	slate	slave	flats	flawn	flaws	flawy	flaxy
snail	stale	stalk	whale	flays	foals	geals	glace	glads
avail	clack	class	elate	glady	glaik	glans	glazy	goals
glass	koala	plaza	slash	gyals	heald	heals	kcal	klang
				klaps	neals	odals	olate	opals
				ovals	peals	plack	plage	plaig

The result in two lists: on the left the words that can still be the solution, on the right those accepted only as a guess. Within each list, words with the most distinct letters come first.

What is missing

A tool I built for myself, not a product. Its limits are listed here so nobody has to go looking for them.

No automated tests; checked by hand against real rounds of the game

The interface sits in a single 305-line file

The program suggests no word, it only filters — a ranking by information gain is missing

Pre-built for Windows only, though it runs anywhere from source

STACK

Python 3	Tkinter	PyInstaller	Git
----------	---------	-------------	-----

DELIVERY

Shipped as a ZIP containing the program folder: unpack and start, no Python needed. Deliberately a folder rather than a single EXE, because a self-extracting executable is regularly flagged as suspicious by virus scanners and browsers.

Download the Windows build ZIP · 13 MB

([https://github.com/KarimBk7/Wordle-](https://github.com/KarimBk7/Wordle-Solver/releases/latest/download/Wordle-Solver-windows.zip)

[Solver/releases/latest/download/Wordle-Solver-windows.zip](https://github.com/KarimBk7/Wordle-Solver/releases/latest/download/Wordle-Solver-windows.zip))

CONTEXT

Private work; the source code is entirely mine. The two word lists are the game's own and come from public collections, credited as such in the repository.

github.com/KarimBk7/Wordle-Solver

(<https://github.com/KarimBk7/Wordle-Solver>)